Assignment 10

Table 1: A router has the following routing table:

Destination Network	Prefix Length	Next Hop
0.0.0.0	/0	198.51.100.1 (default route)
172.20.0.0	/16	10.10.10.2
172.20.4.0	/22	10.10.10.3
172.20.4.128	/25	10.10.10.4
192.0.2.0	/24	directly connected (eth1)

For each destination IP address below, list every entry that matches it, then state which single entry Longest Prefix Match selects and the resulting next hop:

a) 172.20.4.200 b) 172.20.4.5 c)172.20.9.9 d)192.0.2.77 e) 203.0.113.50

Task 1 — Routing Table: LPM Lookup

Starter file: task1_routing_table.py (attached separately). Implement prefix_to_mask() and the add_route() / lookup() methods of RoutingTable, following their docstrings exactly.

- prefix_to_mask(prefix_length) — build a 32-bit subnet mask from a CIDR prefix length.
- RoutingTable.add_route(network, prefix_length, next_hop, label) — store one routing entry.
- RoutingTable.lookup(dest_ip) — apply Longest Prefix Match across all stored routes and return the winning entry's forwarding decision, or None if nothing matches at all.

Running python task1_routing_table.py directly builds the exact same routing table as Table 1 and runs the same five lookups as given, so you can cross-check your hand-worked answers against your code's output. A correct implementation prints "Self-check PASSED." at the end.

Task 2 — Multi-Hop Forwarding Simulation

Starter file: task2_forwarding_simulation.py (attached separately). This file is self-contained — it includes its own working copy of RoutingTable and a ready-made 3-router topology (R1 — R2 — R3, connecting two host subnets); see the diagram in the file's module docstring. Implement trace_path(), following its docstring exactly.

- trace_path(start_router, dest_ip, max_hops) — repeatedly call the current router's lookup(), record each hop, and move to the next router (found via the provided ip_owner mapping) until the packet is delivered, dropped, or exceeds max_hops.

Running python task2_forwarding_simulation.py directly traces a packet from Host A's network (192.168.1.0/24, starting at R1) to a Host B address (192.168.2.77), and prints the hop-by-hop path. A correct implementation prints "Self-check PASSED (R1 -> R2 -> R3 -> delivered)." It then also prints an unchecked reverse-direction trace for you to inspect on your own.